23-单片机 5

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

上次课，讲完时钟的配置之后，问了大家一个问题，也就是我们为什么要把系统时钟配置为 120MHz？大家的回答，有很多朋友都说对了。主要就是我们后续要实现无感 FOC 控制，控制中会用到很多算法，例如位置估算、PI 控制、三角函数计算等等。这些算法呢，会耗费很多 CPU 资源，我们必须确保 CPU 执行指令的速度足够的快，才能在一个 PWM 周期内完成电机控制的核心算法。所以呢，我们做电机控制，一般会把单片机的主频都设置为单片机支持的最高频率。

问：pwm 周期最低要求是多少呢，才能保证 foc 正常？

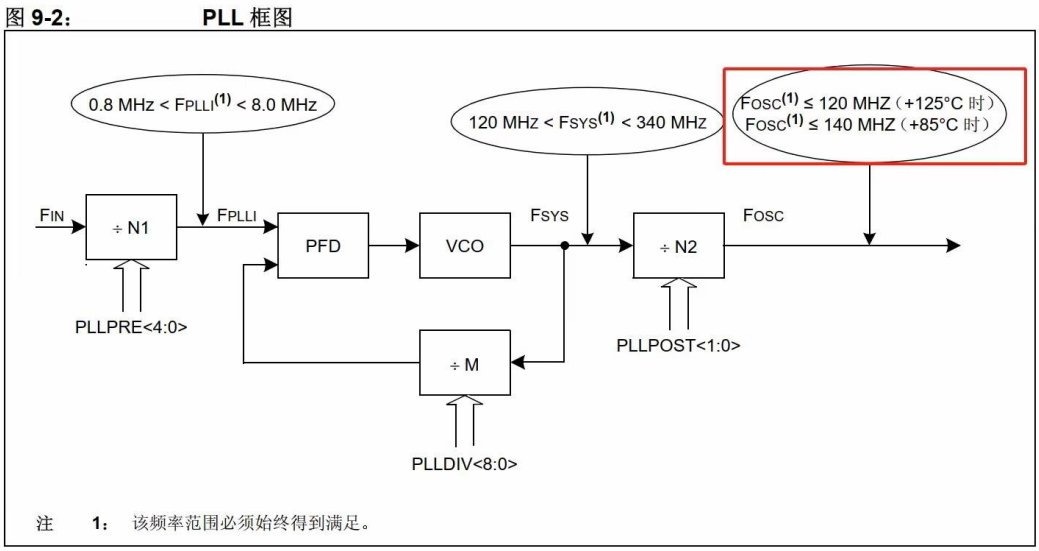
答：PWM 周期要求多少，这个跟主频有一定关系的哈。主频高的话，你的 PWM 周期可以短。一般呢，我们做电机驱动，PWM 载频在 10K~30K 范围内。如果电机转速很高，那么需要的 PWM 载频可能就会更高。PWM 载频越高，对电机的算法执行时间要求就更高，这就需要单片机的主频足够高才行。

像我们课程的无感 FOC 算法，在我们单片机 120MHz 主频下，执行时间大致是 30 多 us。所以呢，针对于我们课程的应用，我们设定 PWM 载频是 20KHz，这样可以确定一个 PWM 周期内把算法执行完毕。

问：单片机主频应该可以做到比 120M 更高，为啥不设定更高的主频？

答：我们单片机比较特别，主频我们设置是 120MHz，PWM 模块的频率是主频的 2 倍。我们单片机的主频最高可以到 140MHz，只不过对温度有要求。

图 9-2：　　PLL 框图



注　1：　该频率范围必须始终得到满足。

大家看这样一幅图，如果我们的应用不超过 85 摄氏度，那么我们其实可以配置主频最高是 140MHz。如果超过的话，就只能设置 120MHz 及以下了。所以呢，我们课程示例，配置的主频是 120MHz。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：20K 的载频，最低要求主频多少？
答：如果全部是 C 语言编写的话，实现无感 FOC，需要 72MHz 及以上。如果是汇编+C 的话，20KHz 载频，48MHz 的单片机可以搞定。我在 M0 单片机上实现过，跑无感 FOC，汇编+C，载频可以设置 20KHz，算法可以跑完的。

问：最低要求主频是测试的还是计算的？
答：实测的。

问：什么是载频？
答：PWM 波的频率。

问：也就是说只要在 pwm 周期内跑完 foc 代码多少频率都无所谓对吧？
答：如果你说的频率是主频的话，是这样。但是如果你说的频率是 PWM 频率的话，PWM 频率我们希望呢，肯定越高越好，但是太高的话，功率管损耗会很大，发热会很严重。所以，如果发热你能接受，那么载频就可以提高。

问：现在芯片可以支持的最高频率一般都高，直接配到顶就好了？
答：一般是的。因为我们做电机驱动的话，不怎么考虑低功耗，所以可以把主频设置到最高。

问：十万转要多大的频率？
答：十万转的话，我们风筒课程，设置载频是 25KHz，主频是 170MHz。

问：我们说 10-30k 的 pwm 频率实际是根据常用单片机的主频 40-120 甚至更高得来吗？
答：实际使用经验，以及功率管发热，算法执行时间等得出的。

问：老师假如 10 转以下哪？主频多大？
答：要实际匹配哈，170MHz 以内，应该可以搞定。

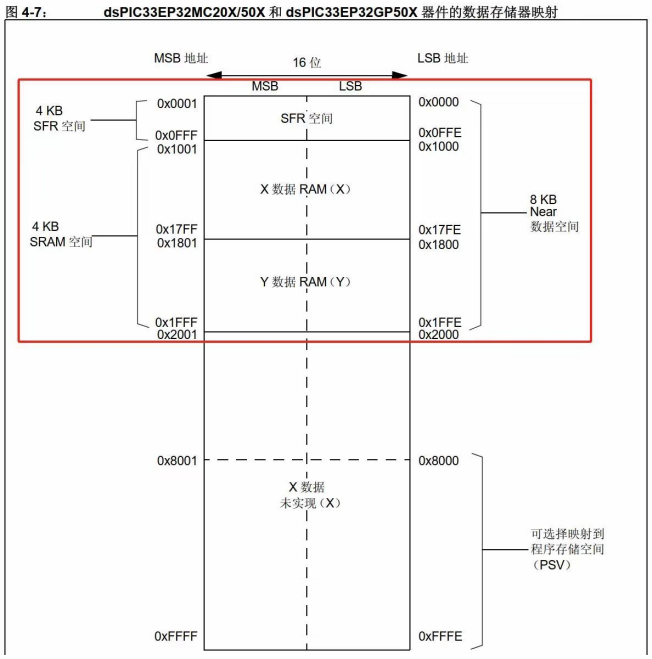
问：十万转是电频率还是机械？
答：高速电机，极对数一般很少，比如吹风机中使用的，是一对极，那么 10 万转既是电转速也是机械转速。如果是其他极对数的话，我们一般称电机可以到多少转速，都是指的机械转速。

好，关于主频我们就讨论这么多了哈，下面我们接着来分析启动文件中剩余的代码。

```
45
46      mov     #0x1000,W0
47      mov     #0x2000,W1
48      clr     [W0++]
49      xor     W0,W1,W2
50      bra     NZ,$-4
51
52      goto    _StartProgram
53
54      .end
55  /*============================Copyright belongs to ZhangFeiShiZhanDianZi=====
56
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

时钟配置好之后，红框框出来的这段代码是 RAM 清零的操作。RAM 清零之后，也就是说，我们定义的变量的初值都会是 0。那么，要对 RAM 清零的话，我们需要知道 RAM 的起始地址及结束地址。然后呢，就从起始地址开始，往地址里面写 0，一直写到结束地址后，清零这个动作才完成。



从单片机的手册中，我们可以知道我们使用的 dsPIC33EP32MC204 型号的 RAM 空间范围。可用的 RAM 空间范围，是从 0x1000 到 0x2000。

问：为什么这里有个 xy 的区分？

答：其实常规指令应用，x 跟 y 是组成一个整体使用的。有一些特殊的指令，它可以在访问一个地址的数据时顺带访问其它地址的数据。也就是一次操作，可以访问多个数据。此时就把 RAM 空间当成 x　y 来使用了。

MAC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd, AWB	相乘并累加

比如说，这样的指令。它可以实现相乘累加功能的同时，再分别从 x 地址，y 地址取出数据。

好，我们接着来看啊。我们清零的话，就是清零从地址 0x1000 开始到 0x2000 结束这段范围内的空间。

```
mov     #0x1000,W0
mov     #0x2000,W1
clr     [W0++]
xor     W0,W1,W2
bra     NZ,$-4
```

因此呢，我们就分别把起始地址跟结束地址分别放到了 w0 跟 w1 寄存器中暂存起来。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
mov     #0x1000, W0
mov     #0x2000, W1
clr     [W0++]
xor     W0, W1, W2
bra     NZ, $-4
```

然后呢，使用 clr(清零指令)对地址中的内容进行清零。w0++的话，是指令执行后，然后 w0 中存储的数据+2。也就是第一次执行 clr 的时候，w0 中存储的是 0x1000，执行后，w0 中存储的是 0x1002。然后呢，我们需要判断是否到了结束地址。

```
mov     #0x1000, W0
mov     #0x2000, W1
clr     [W0++]
xor     W0, W1, W2
bra     NZ, $-4
```

这里的判断呢，使用了异或的方法。异或的话，就是相同为 0，不同为 1。也就是判断 w0 跟 w1 中的存储的数据是否相同，如果相同那么 Z 标志位就是 1，如果不同 Z 标志位就是 0。所以呢，异或操作执行后，我们可以根据 Z 标志位来得知 W0 是否跟 W1 存储的数据相等。如果 Z=1，那么说明两者相等了，则说明清零就结束了。如果说，Z=0，说明两者还不相等，则说明清零还没结束，还需要继续执行清零动作。

```
mov     #0x1000, W0
mov     #0x2000, W1
clr     [W0++]
xor     W0, W1, W2
bra     NZ, $-4
```

所以呢，这里通过 bra 条件跳转指令，结合 Z 标志位来判断是否需要跳转。NZ 的意思是 Z=0。也就是 Z=0，则会跳转。

问：W2 起啥作用？
答：在这里，没有用，只是指令需要。

问：清零的作用是什么？不清行不行？
答：因为 RAM 中的数据是随机的，而我们写代码的时候，会定义一些变量，那么这些变量可能数据就是随机的，这样的话，我们就需要自己手动把这些变量清 0。这里的话，我们是统一操作了，把 RAM 空间都清零了。后续就不需要对变量设置清零了。我们习惯是先对 RAM

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

进行清零操作。大家也可以不清零，然后自己到 C 文件中定义初始化函数，进行清零。

问：是不是大部分指令对 RAM 的操作都是两个字节的？
答：这个指令有说明的。

汇编语法		说明	字数	周期数	页码
EXCH	Wns,Wnd	交换 Wns 和 Wnd 的内容	1	1	243
MOV	f {,WREG}(1)	将 f 中内容传送至目标寄存器	1	1	279
MOV	WREG,f	将 WREG 中内容传送至 f	1	1	280
MOV	f,Wnd	将 f 中内容传送至 Wnd	1	1(4)	281
MOV	Wns,f	将 Wns 中内容传送至 f	1	1	282
MOV.B	#lit8,Wnd	将 8 位立即数传送至 Wnd	1	1	283
MOV	#lit16,Wnd	将 16 位立即数传送至 Wnd	1	1	284
MOV	[Ws+Slit10],Wnd	将 [Ws + 有符号 10 位偏移量] 中内容传送至 Wnd	1	1(4)	285
MOV	Wns,[Wd+Slit10]	将 Wns 中内容传送至 [Wd + 有符号 10 位偏移量]	1	1	286
MOV	Wso,Wdo	将 Wso 中内容传送至 Wdo	1	1(4)	287
MOV.D	Ws,Wnd	双字传送 Ws 内容至 Wnd:Wnd + 1	1	2(4)	289
MOV.D	Wns,Wd	双字传送 Wns:Wns + 1 内容至 Wd	1	2	289
MOVPAG	#lit10,DSRPAG(2)	将 10 位立即数传送至 DSRPAG	1	1	291
MOVPAG	#lit9,DSWPAG(2)	将 9 位立即数传送至 DSWPAG	1	1	291
MOVPAG	#lit8,TBLPAG(2)	将 8 位立即数传送至 TBLPAG	1	1	291
MOVPAG	Wn, DSRPAG(2)	将 Wn 中内容传送到 DSRPAG	1	1	292
MOVPAG	Wn, DSWPAG(2)	将 Wn 中内容传送到 DSWPAG	1	1	292
MOVPAG	Wn, TBLPAG(2)	将 Wn 中内容传送到 TBLPAG	1	1	292
SWAP	Wn	Wn = Wn 内容字节或半字节交换	1	1	426
TBLRDH	[Ws],Wd	将高位程序字读入 Wd	1	2(3)	427
TBLRDL	[Ws],Wd	将低位程序字读入 Wd	1	2(3)	429
TBLWTH	Ws,[Wd]	将 Ws 中的内容写入高位程序字	1	2(4)	431
TBLWTL	Ws,[Wd]	将 Ws 中的内容写入低位程序字	1	2(4)	433

你看这样的指令表，如果指令后面带了.B 或.b，则对应的是字节操作。如果带了.w 或者省略的，就是字操作。如果是带了.d 或者.D 的，就是双字操作。一般呢，我们默认不带后面的.b .d .w 就默认是字操作。

好的，我们继续。

```
mov     #0x1000,W0
mov     #0x2000,W1
clr     [W0++]
xor     W0,W1,W2
bra     NZ,$-4

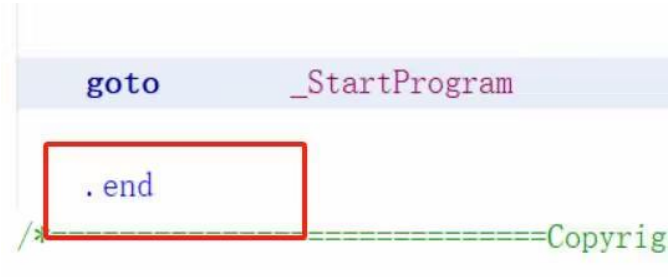
goto    _StartProgram

.end
/*============================Copyright be
```

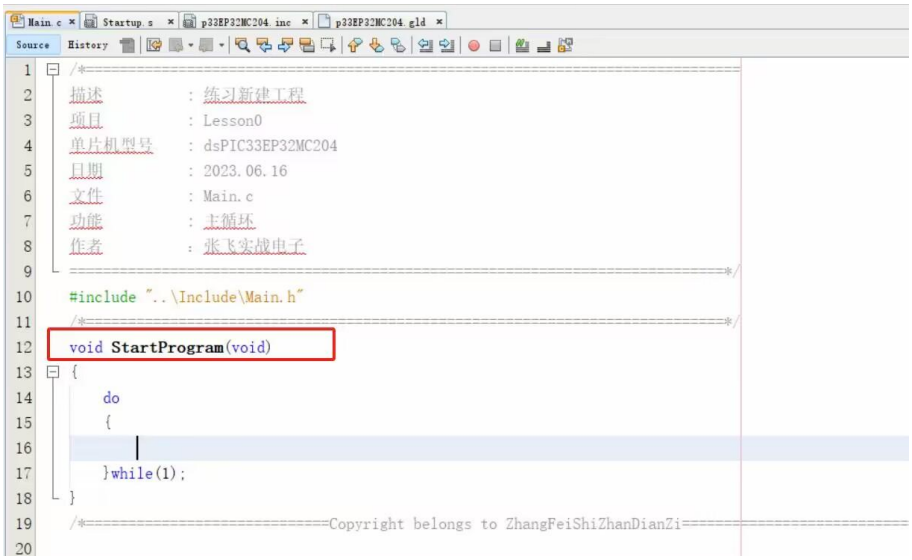
启动文件的代码执行结束后，就会使用一条 GOTO 指令跳转到主程序。这个 StartProgram 就是主函数的名字。

问：goto 是 c 语言还是汇编语言？
答：.s 文件中的都是汇编语言。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



最后，汇编文件结束的话，需要加上.end。这样的话，启动文件就分析完了。



也就是说，启动文件执行后，通过跳转指令，就把代码跳转到主函数这边来了。后续呢，代码就从主函数往下执行。这是从单片机刚上电复位后，从启动文件的复位向量开始跳转到主程序的执行流程。这个过程，主要是通过启动文件来实现的。

那么，在分析启动文件过程中呢，涉及到了一些汇编语言，接触到了一些汇编指令。这里呢，希望大家不要害怕汇编，只要我们有思路，汇编就是指令的问题，指令的话，我们又不需要去死背，用的时候知道到什么地方去查就行。

所以呢，关于启动文件，我们就讲这么多了。接下来呢，会再给大家介绍 IO 模块 PWM 模块 ADC 模块。至于其他的模块呢，大家可以后续自己学习一下。

关于启动文件，大家还有疑问没？

问：启动文件有没有向量表？
答：我们单片机没有，向量表在 gld 链接描述文件中。

好，那接下来我们来讲 IO 模块。

IO 端口模块的话，其实就是设置 IO 端口是数字口还是模拟口。数字口的话，就是设置数字输入，或数字输出。而模拟口的话，主要是用于模拟信号的采样或者模拟信号的输出。例如，后续我们要讲的 ADC 模块，就是对输入的模拟信号采样，然后转换为数字信号。而如果单片机有 DAC 模块的话，就是把数字信号变为模拟电压信号输出。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

数字口的话，一般分为普通的数字 IO 口及外设复用的数字 IO 口。普通的数字 IO，就是让 IO 输出高电平或者低电压，或者读外部引脚的电平信号。例如控制 LED 灯，一般就是普通的数字 IO 输出功能。读外部按键的状态，就是普通的数字 IO 输入功能。所以说，我们要使用 IO 模块的话，无非就是对输入输出，模拟数字，普通或复用功能的配置。

这些配置呢，都涉及到对应的寄存器。我们可以从手册中的特殊功能寄存器映射表查看 IO 端口的寄存器。



大家可以打开单片机的数据手册，找到表 4-53，可以知道这个单片机的 IO 模块汇总的寄存器列表。

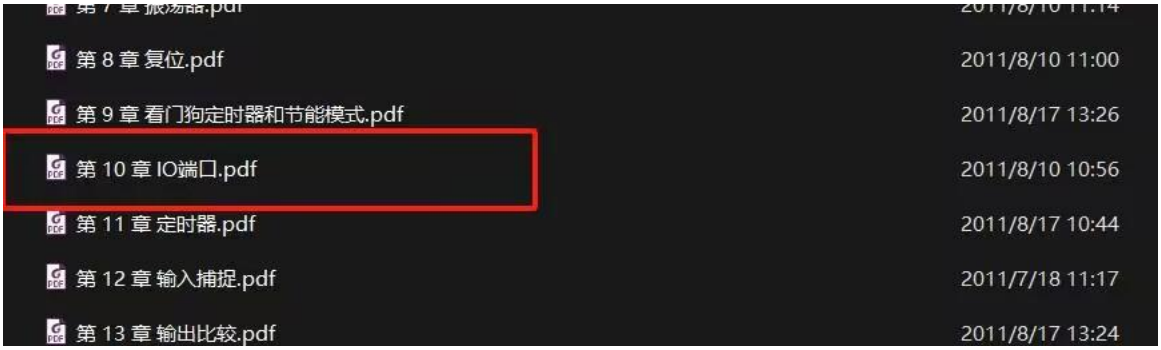
表 4-53： PORTA 寄存器映射 （仅限 PIC24EPXXXGP/MC204 和 dsPIC33EPXXXGP/MC204/504 器件）

寄存器名称	地址	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	所有复位时的状态
TRISA	0E00	—	—	—	—	—	TRISA10	TRISA9	TRISA8	TRISA7	—	—	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	079F
PORTA	0E02	—	—	—	—	—	RA10	RA9	RA8	RA7	—	—	RA4	RA3	RA2	RA1	RA0	0000
LATA	0E04	—	—	—	—	—	LATA10	LATA9	LATA8	LATA7	—	—	LATA4	LATA3	LATA2	LA1TA1	LA0TA0	0000
ODCA	0E06	—	—	—	—	—	ODCA10	ODCA9	ODCA8	ODCA7	—	—	ODCA4	ODCA3	ODCA2	ODCA1	ODCA0	0000
CNENA	0E08	—	—	—	—	—	CNIEA10	CNIEA9	CNIEA8	CNIEA7	—	—	CNIEA4	CNIEA3	CNIEA2	CNIEA1	CNIEA0	0000
CNPUA	0E0A	—	—	—	—	—	CNPUA10	CNPUA9	CNPUA8	CNPUA7	—	—	CNPUA4	CNPUA3	CNPUA2	CNPUA1	CNPUA0	0000
CNPDA	0E0C	—	—	—	—	—	CNPDA10	CNPDA9	CNPDA8	CNPDA7	—	—	CNPDA4	CNPDA3	CNPDA2	CNPDA1	CNPDA0	0000
ANSELA	0E0E	—	—	—	—	—	—	—	—	—	—	—	ANSA4	—	—	ANSA1	ANSA0	0013

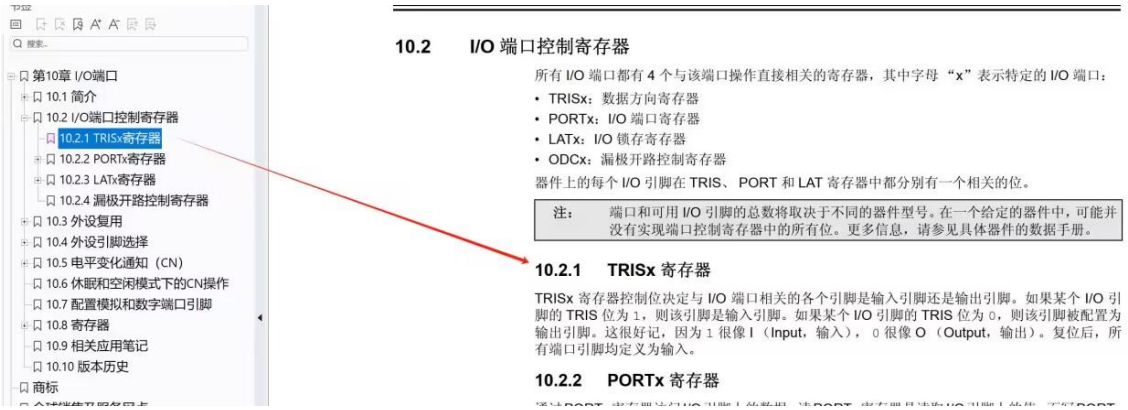
图注： — = 未实现，读为 0。复位值以十六进制显示。

例如，我们以 PORTA 为例，来看下这些寄存器的功能。TRISA，这个寄存器就是用于设置 A 口各个引脚的方向的。对应的这些位设置为 1，引脚就是输入功能，设置为 0，引脚就是输出功能。也就是 TRISA 是方向寄存器。

问：这个手册哪里能看到这个寄存器的说明？
答：要详细说明的话，需要到分章节的手册中去找。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

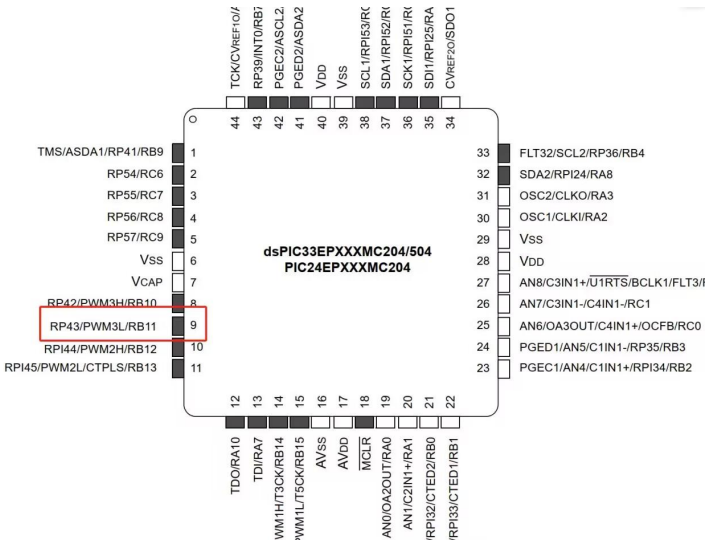


然后 PORTA 这个寄存器一般用于读取引脚的电平信号。也就是数字输入的时候，可以使用这个寄存器。LATA 寄存器，是普通数字输出的时候，给引脚写电平状态。ODCA 是引脚开漏功能的设置。然后，CNENA 是引脚中断的使能设置寄存器。CNPUA 及 CNPDA 分别是引脚的弱上拉及下拉功能的设置。ANSELA 是引脚数字功能跟模拟功能选择的配置寄存器。那么，整个 IO 模块，涉及到的就是就是这些寄存器。

我们实际使用的时候，根据需要进行配置。并不是所有的寄存器我们都会用上，选择跟我们功能实现相关的寄存器就行。那么，关于其他口（B/C）的寄存器都是一样的。大家了解了 A 口的，其他口的使用方法是一致的。大家看下，关于这些寄存器的功能介绍，大家有没有疑问？

问：复用功能是设置哪个寄存器？

答：我们单片机的话，如果外设的引脚是固定的，那么复用功能的使用就是直接使能对应的外设就配置好了。如果说外设引脚是可以配置的，那么复用功能的使用就需要搭配外设功能选择寄存器及外设模块的配置来使能。



可以看下我们单片机的这个引脚图。这个引脚的功能优先级是从左到右的。RPxx 这样的引脚，就是外设复用可以选用的引脚。PWM3L 这样的引脚，就是外设专用引脚。RB11 这样的标号，就是普通的 IO 口。所以，如果我们使用了外设复用功能配置的话，那么这个引脚就会优先作为 RPxx 使用。如果没有设置 RPxx 功能的话，引脚的使用就是下一个优先级，如果下一个还不使用，那么就是普通的 IO 功能。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

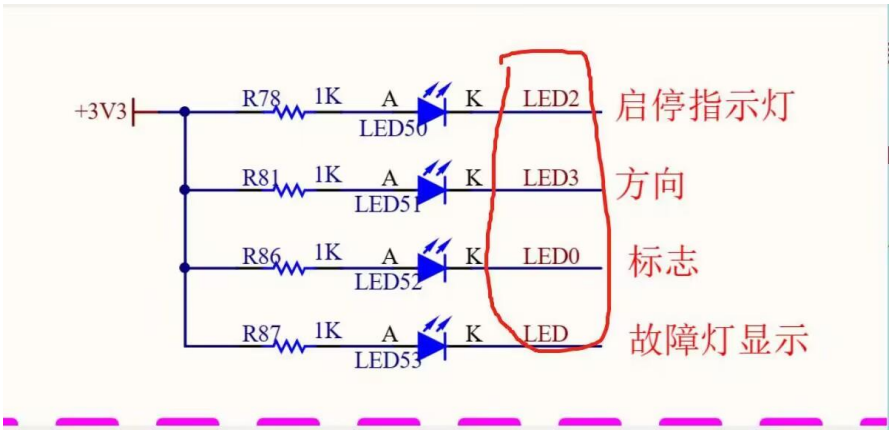
问：输出状态的时候，能读 PORTA 得到 IO 状态不？
答：可以读。

问：怎么设置使用和不使用呢？
答：如果是外设要使用这个引脚，那么就是配置好外设的寄存器，使能外设模块，外设就获得了这个引脚的使用权。例如，我们要使用 PWM3L 这个引脚输出 PWM 波，则我们配置好 PWM 模块，使能它，这个引脚就属于这个模块引脚了。如果你要是用这个引脚的普通 IO 功能，那么 PWM 模块就不能占用它。RPxx 功能也不能占用它。

好，那么我们下面就给大家分析一下普通数字输出 IO 功能的使用代码。



我们板子上呢，有 4 颗 LED 灯。这四颗 LED 灯都是连接到 A 口的。分别是 A2　A3　A7　A10。

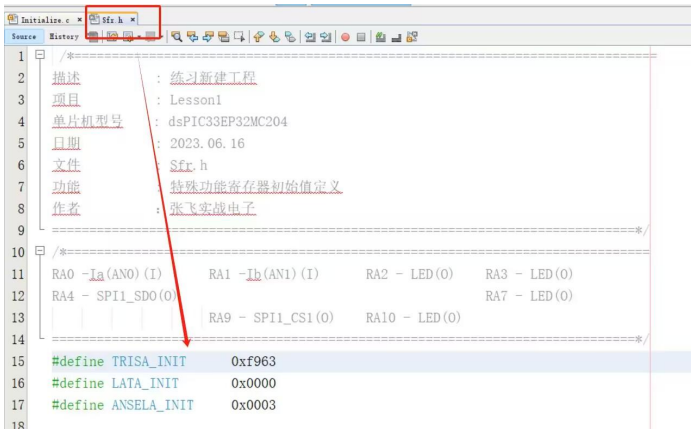


然后，根据 LED 灯的连接电路，我们可以知道，给 IO 口输出低电平，就可以让 LED 灯点亮，输出高电平，就可以让 LED 灯熄灭。所以说，这里我们主要涉及到三个寄存器的操作。一个是 TRISA　设置 IO 口的方向，设置为输出。另一个是　LATA，设置 IO 口的输出电平。最后是 ANSELA，设置 IO 口是数字口还是模拟口，这里我们需要设置为数字口。

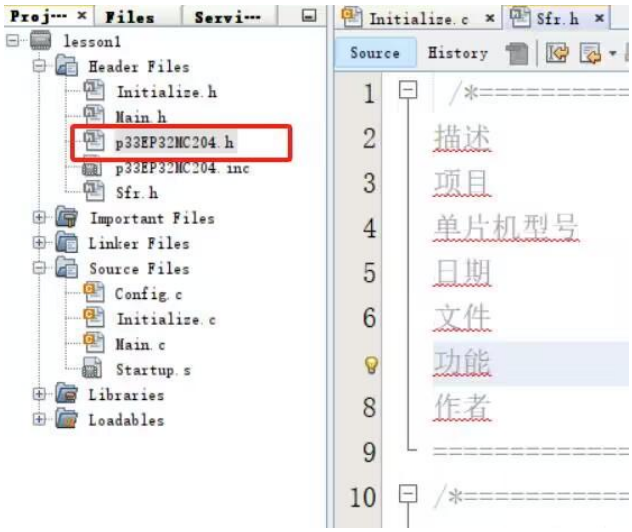
```
void Initialize(void)
{
    TRISA = TRISA_INIT;
    LATA = LATA_INIT;
    ANSELA = ANSELA_INIT;
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

所以，这里写了一个初始化函数，用于配置这三个寄存器。



这三个寄存器的初值呢，是用宏定义设置的。



我们引用这个官方头文件，就有寄存器的定义。

```
#define TRISA TRISA
extern volatile unsigned int  TRISA __attribute__((__sfr__));
typedef struct tagTRISABITS {
  unsigned TRISA0:1;
  unsigned TRISA1:1;
  unsigned TRISA2:1;
  unsigned TRISA3:1;
  unsigned TRISA4:1;
  unsigned :2;
  unsigned TRISA7:1;
  unsigned TRISA8:1;
  unsigned TRISA9:1;
  unsigned TRISA10:1;
} TRISABITS;
extern volatile TRISABITS TRISAbits __attribute__((__sfr__));
```

打开头文件之后，可以搜索这个寄存器的名字，就可以找到它的定义了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

表 4-53： PORTA 寄存器映射（仅限 PIC24EPXXXGP/MC204 和 dsPIC33EPXXXGP/MC204/504 器件）

寄存器名称	地址	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	所有复位时的状态
TRISA	0E00	—	—	—	—	—	TRISA10	TRISA9	TRISA8	TRISA7	—	—	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	079F
PORTA	0E02	—	—	—	—	—	RA10	RA9	RA8	RA7	—	—	RA4	RA3	RA2	RA1	RA0	0000
LATA	0E04	—	—	—	—	—	LATA10	LATA9	LATA8	LATA7	—	—	LATA4	LATA3	LATA2	LA1TA1	LA0TA0	0000
ODCA	0E06	—	—	—	—	—	ODCA10	ODCA9	ODCA8	ODCA7	—	—	ODCA4	ODCA3	ODCA2	ODCA1	ODCA0	0000
CNENA	0E08	—	—	—	—	—	CNIEA10	CNIEA9	CNIEA8	CNIEA7	—	—	CNIEA4	CNIEA3	CNIEA2	CNIEA1	CNIEA0	0000
CNPUA	0E0A	—	—	—	—	—	CNPUA10	CNPUA9	CNPUA8	CNPUA7	—	—	CNPUA4	CNPUA3	CNPUA2	CNPUA1	CNPUA0	0000
CNPDA	0E0C	—	—	—	—	—	CNPDA10	CNPDA9	CNPDA8	CNPDA7	—	—	CNPDA4	CNPDA3	CNPDA2	CNPDA1	CNPDA0	0000
ANSELA	0E0E	—	—	—	—	—	—	—	—	—	—	—	ANSA4	—	—	ANSA1	ANSA0	0013

图注： — = 未实现，读为 0。复位值以十六进制显示。

实际上，如果我们只配置 LED 灯的 IO 口的话，只需要把 A2　A3　A7　A10 清 0 即可。但是这里结合原理图，我们设置的时候，把 A 口对应的方向位都设置了。也就是 A0　是模拟输入 A1 也是模拟输入，A4 是数字输出，A9 是输出输出。因此呢，就顺便把这些口设置了。然后，没有使用的引脚呢，我们全部设置为 1 了。也就是 A0　是模拟输入　A1 也是模拟输入，A4 是数字输出，A9 是数字输出。

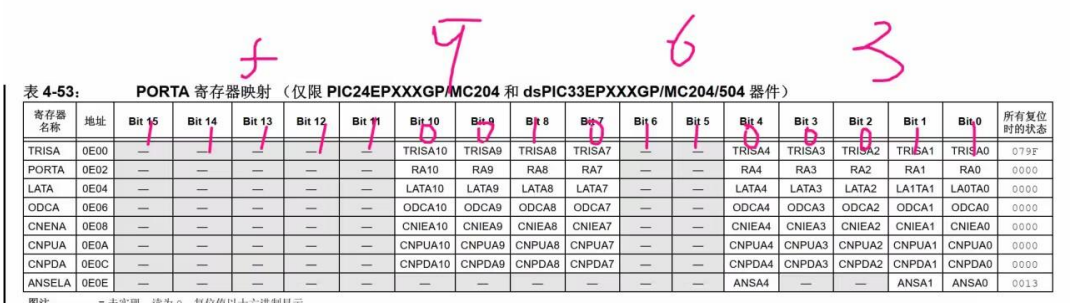


表 4-53： PORTA 寄存器映射（仅限 PIC24EPXXXGP/MC204 和 dsPIC33EPXXXGP/MC204/504 器件）

寄存器名称	地址	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	所有复位时的状态
TRISA	0E00	—	—	—	—	—	TRISA10	TRISA9	TRISA8	TRISA7	—	—	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	079F
PORTA	0E02	—	—	—	—	—	RA10	RA9	RA8	RA7	—	—	RA4	RA3	RA2	RA1	RA0	0000
LATA	0E04	—	—	—	—	—	LATA10	LATA9	LATA8	LATA7	—	—	LATA4	LATA3	LATA2	LA1TA1	LA0TA0	0000
ODCA	0E06	—	—	—	—	—	ODCA10	ODCA9	ODCA8	ODCA7	—	—	ODCA4	ODCA3	ODCA2	ODCA1	ODCA0	0000
CNENA	0E08	—	—	—	—	—	CNIEA10	CNIEA9	CNIEA8	CNIEA7	—	—	CNIEA4	CNIEA3	CNIEA2	CNIEA1	CNIEA0	0000
CNPUA	0E0A	—	—	—	—	—	CNPUA10	CNPUA9	CNPUA8	CNPUA7	—	—	CNPUA4	CNPUA3	CNPUA2	CNPUA1	CNPUA0	0000
CNPDA	0E0C	—	—	—	—	—	CNPDA10	CNPDA9	CNPDA8	CNPDA7	—	—	CNPDA4	CNPDA3	CNPDA2	CNPDA1	CNPDA0	0000
ANSELA	0E0E	—	—	—	—	—	—	—	—	—	—	—	ANSA4	—	—	ANSA1	ANSA0	0013

所以，根据功能设置，这个方向寄存器的初值就定了。

```
/*==========================================================================
RA0 -Ia(AN0)(I)      RA1 -Ib(AN1)(I)      RA2 - LED(0)      RA3 - LED(0)
RA4 - SPI1_SDO(0)                                            RA7 - LED(0)
                     RA9 - SPI1_CS1(0)    RA10 - LED(0)
==========================================================================*/

#define TRISA_INIT      0xf963
#define LATA_INIT       0x0000
#define ANSELA_INIT     0x0003
```

然后 LATA 寄存器的话，这里我们设置的是全 0。也就是说，这样上电，四颗 LED 灯会常亮。如果希望 LED 灯熄灭的话，我们可以把 A2　A3　A7　A10 设置为 1。

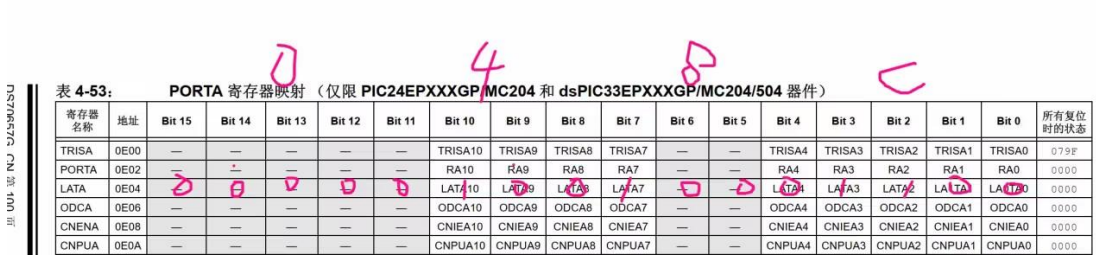


表 4-53： PORTA 寄存器映射（仅限 PIC24EPXXXGP/MC204 和 dsPIC33EPXXXGP/MC204/504 器件）

寄存器名称	地址	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	所有复位时的状态
TRISA	0E00	—	—	—	—	—	TRISA10	TRISA9	TRISA8	TRISA7	—	—	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	079F
PORTA	0E02	—	—	—	—	—	RA10	RA9	RA8	RA7	—	—	RA4	RA3	RA2	RA1	RA0	0000
LATA	0E04	—	—	—	—	—	LATA10	LATA9	LATA8	LATA7	—	—	LATA4	LATA3	LATA2	LA1TA1	LA0TA0	0000
ODCA	0E06	—	—	—	—	—	ODCA10	ODCA9	ODCA8	ODCA7	—	—	ODCA4	ODCA3	ODCA2	ODCA1	ODCA0	0000
CNENA	0E08	—	—	—	—	—	CNIEA10	CNIEA9	CNIEA8	CNIEA7	—	—	CNIEA4	CNIEA3	CNIEA2	CNIEA1	CNIEA0	0000
CNPUA	0E0A	—	—	—	—	—	CNPUA10	CNPUA9	CNPUA8	CNPUA7	—	—	CNPUA4	CNPUA3	CNPUA2	CNPUA1	CNPUA0	0000

也可以设置为这样，上电 LED 是熄灭的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================================
RA0 -Ia(AN0)(I)      RA1 -Ib(AN1)(I)      RA2 - LED(O)     RA3 - LED(O)
RA4 - SPI1_SDO(O)                                          RA7 - LED(O)
                     RA9 - SPI1_CS1(O)    RA10 - LED(O)
==========================================================================*/
#define TRISA_INIT      0xf963
#define LATA_INIT       0x0000
#define ANSELA_INIT     0x0003

/*==========================版权归张飞实战电子所有，违者必究！！！==========*/
```

然后，最后一个是模拟数字功能的设置。因为 AOA1 是模拟口，需要把对应的位设置 1，其他的位都是 0。

表 4-53： PORTA 寄存器映射（仅限 PIC24EPXXXGP/MC204 和 dsPIC33EPXXXGP/MC204/504 器件）

寄存器名称	地址	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	所有复位时的状态
TRISA	0E00	—	—	—	—	—	TRISA10	TRISA9	TRISA8	TRISA7	—	—	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	079F
PORTA	0E02	—	—	—	—	—	RA10	RA9	RA8	RA7	—	—	RA4	RA3	RA2	RA1	RA0	0000
LATA	0E04	—	—	—	—	—	LATA10	LATA9	LATA8	LATA7	—	—	LATA4	LATA3	LATA2	LA1TA1	LA0TA0	0000
ODCA	0E06	—	—	—	—	—	ODCA10	ODCA9	ODCA8	ODCA7	—	—	ODCA4	ODCA3	ODCA2	ODCA1	ODCA0	0000
CNENA	0E08	—	—	—	—	—	CNIEA10	CNIEA9	CNIEA8	CNIEA7	—	—	CNIEA4	CNIEA3	CNIEA2	CNIEA1	CNIEA0	0000
CNPUA	0E0A	—	—	—	—	—	CNPUA10	CNPUA9	CNPUA8	CNPUA7	—	—	CNPUA4	CNPUA3	CNPUA2	CNPUA1	CNPUA0	0000
CNPDA	0E0C	—	—	—	—	—	CNPDA10	CNPDA9	CNPDA8	CNPDA7	—	—	CNPDA4	CNPDA3	CNPDA2	CNPDA1	CNPDA0	0000
ANSELA	0E0E	—	—	—	—	0	0	0	0	0	0	0	ANSA4	0	0	ANSA1	ANSA0	0013

图注： — = 未实现，读为 0。复位值以十六进制显示。

表 4-54： PORTB 寄存器映射（仅限 PIC24EPXXXGP/MC204 和 dsPIC33EPXXXGP/MC204/504 器件）

所以，这样就完成了 4 颗 LED 灯控制需要的寄存器配置。

```
9   /*==========================================================================
10  #include "..\Include\Main.h"
11  /*==========================================================================
12  void StartProgram(void)
13  {
14      Initialize();
15      do
```

然后，我们调用一下这个初始化函数，寄存器配置就生效了。

```
void StartProgram(void)
{
    Initialize();
    do
    {
        LATAbits.LATA2 = 0;
        LATAbits.LATA3 = 0;
        LATAbits.LATA7 = 0;
        LATAbits.LATA10 = 0;
        Delay();
        LATAbits.LATA2 = 1;
        LATAbits.LATA3 = 1;
        LATAbits.LATA7 = 1;
        LATAbits.LATA10 = 1;
        Delay();
    }while(1);
}
```

然后呢，我们在主循环中操控对应的引脚，让四个引脚拉低，间隔一段时间，然后拉高，再间隔一段时间。这样可以实现一个简单的 LED 灯闪烁功能。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：老师，delay()什么作用?
答：控制 LED 灯闪烁的。

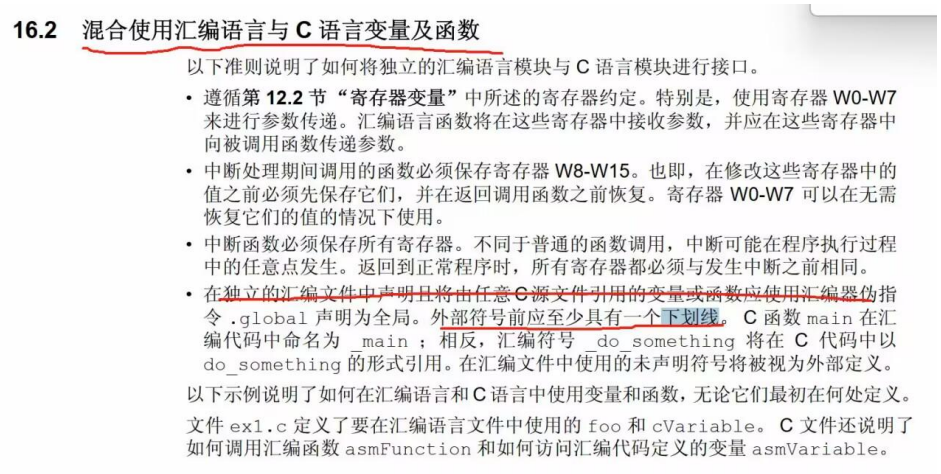
```
/*=====================================================================*/
void Delay(void)
{
    unsigned long i;

    i = 3276800;
    while(i-- != 0);
}
```

这里是用循环实现的一个简单延时。目的是给大家示意一下 IO 口输出功能的配置。那么，关于 IO 端口的使用，我们就分析这么多了。大家可以自己下去抽时间练习一下。

问：GOTO 和那个 bra 有什么区别呀?
答：GOTO 是无条件跳转，BRA 是有条件的。

问：`goto _StartProgram` 这个为什么加一个横线？
答：这是因为 C 编译器在编译的时候，需要在标识符前加上下划线。也就是编译器的规定，我们遵循即可。



16.2 混合使用汇编语言与 C 语言变量及函数

以下准则说明了如何将独立的汇编语言模块与 C 语言模块进行接口。

- 遵循第 12.2 节“寄存器变量”中所述的寄存器约定。特别是，使用寄存器 W0-W7 来进行参数传递。汇编语言函数将在这些寄存器中接收参数，并应在这些寄存器中向被调用函数传递参数。
- 中断处理期间调用的函数必须保存寄存器 W8-W15。也即，在修改这些寄存器中的值之前必须先保存它们，并在返回调用函数之前恢复。寄存器 W0-W7 可以在无需恢复它们的值的情况下使用。
- 中断函数必须保存所有寄存器。不同于普通的函数调用，中断可能在程序执行过程中的任意点发生。返回到正常程序时，所有寄存器都必须与发生中断之前相同。
- 在独立的汇编文件中声明且将由任意 C 源文件引用的变量或函数应使用汇编器伪指令 .global 声明为全局。外部符号前应至少具有一个下划线。C 函数 main 在汇编代码中命名为 _main；相反，汇编符号 _do_something 将在 C 代码中以 do_something 的形式引用。在汇编文件中使用的未声明符号将被视为外部定义。

以下示例说明了如何在汇编语言和 C 语言中使用变量和函数，无论它们最初在何处定义。

文件 ex1.c 定义了要在汇编语言文件中使用的 foo 和 cVariable。C 文件还说明了如何调用汇编函数 asmFunction 和如何访问汇编代码定义的变量 asmVariable。

问：单个指令周期，是 2 倍主频的倒数吗?
答：1 个指令周期 = 1/Fcy = 1/Fosc/2 = 1/60MHz。

好了，那我们今天就先讲到这里。大家早点休息，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。